

Getting started

Da i prerequisiti al simulatore avviato!

Prerequisiti

1. CPU con almeno 8 core
2. GPU **NVIDIA possibilmente con almeno 4 GB di vRAM dedicata**
3. 8 GB di RAM
4. Ubuntu 22.04 Jammy Jollyfish (installato nativo) {NO VM/WSL}

1) Installare DOCKER

Seguendo la guida della [DIGITAL OCEAN](#):

```
sudo apt update
sudo apt install apt-transport-https ca-certificates curl sof
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sud
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/s
sudo apt update
sudo apt install docker-ce

#To execute docker without sudo [RECOMMENDED]
sudo usermod -aG docker ${USER}
su - ${USER}
```

2) Installare il supporto Nvidia per i container (accelerazione grafica)

ACHTUNG! Prima di procedere, controlla che il pc riconosca/veda la scheda video. Per farlo prova a lanciare il comando `nvidia-smi` su un terminale {solitamente si apre con `CTRL+T` e dovresti vedere qualcosa del genere:

```
edo@edo-ThinkPad-P1-Gen-5:~$ nvidia-smi
```

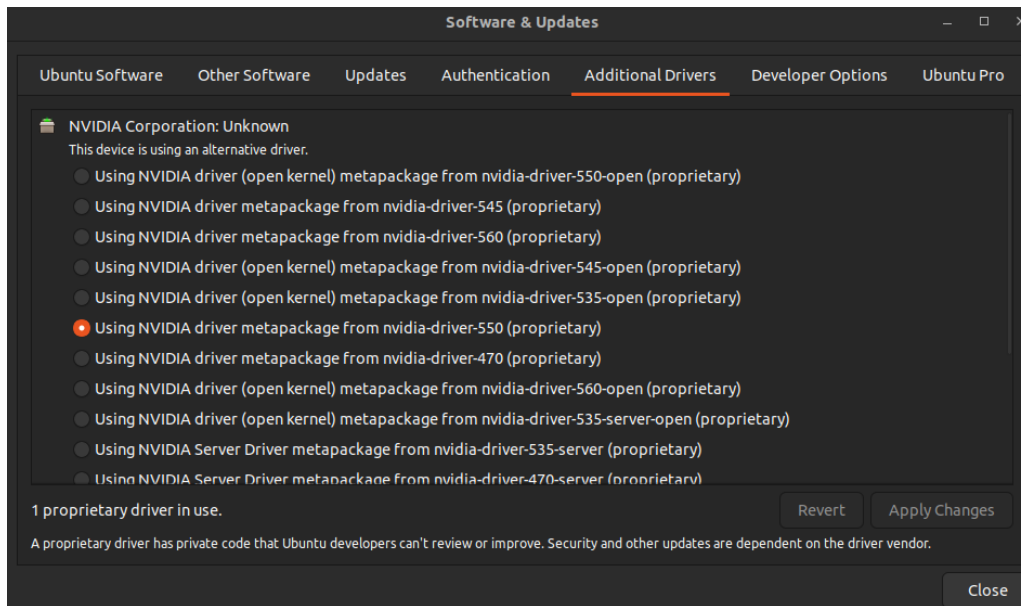
```
Fri Nov 15 02:05:06 2024
```

```
+-----+
| NVIDIA-SMI 550.120                        Driver Version: 550.120
|-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        D
| Fan   Temp     Perf           Pwr:Usage/Cap |      Memory-
|
|=====+=====+
|    0   NVIDIA RTX A2000 8GB Lap...      Off |   00000000:01:00.
| N/A    48C      P3              749W /   35W |          9MiB /  81
|
|-----+-----+

+-----+
| Processes:
|  GPU   GI    CI          PID    Type   Process name
|          ID    ID
|=====+=====+
|    0   N/A   N/A         1862      G   /usr/lib/xorg/Xorg
|-----+-----+
```

Se invece non vedi niente o compare un messaggio del tipo `nvidia-smi command not found` oppure ancora `NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver. Make sure that the latest NVIDIA driver is installed and running.`

Per risolvere prova a premere il tasto `WINDOWS` e digita nella barra di ricerca `additional drivers` che ti rimanderà ad una schermata del genere:



Se vedi tutti questi driver anche tu, seleziona quello **550-proprietary** (se a me funziona dovrebbe funzionare anche a te 🤔) altrimenti, se non vedi niente prova a scrivere su un terminale:

```
sudo apt install nvidia-driver-550
sudo reboot #ti riavvierà il pc
sudo apt update
```

Se neanche così hai risolto, prova a cercare online o [scrivimi](#).

BEEEEENEEEEE ora che sei arrivato a questo punto significa che il tuo pc ti vede la scheda video e siamo tutti contenti.

Dunque, ora puoi installare l' ***Nvidia container toolkit***!

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey
&& curl -s -L https://nvidia.github.io/libnvidia-container/
sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nv
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

```
nvidia-ctk runtime configure --runtime=docker --config=$HOME/
systemctl --user restart docker
sudo nvidia-ctk config --set nvidia-container-cli.no-cgroups
```

Per vedere se hai fatto tutto bene prova a lanciare questo comando: `sudo docker`

```
run --rm --runtime=nvidia --gpus all ubuntu nvidia-smi
```

Le cose sono due:

1. Ti esce questo:

```
+-----+
| NVIDIA-SMI 535.86.10      Driver Version: 535.86.10      CUDA V
|-----+-----+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Vol
| Fan   Temp   Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-
|-----+-----+
| 0     Tesla T4              On          | 00000000:00:1E.0 Off  |
| N/A   34C    P8          9W / 70W | 0MiB / 15109MiB |
|-----+-----+

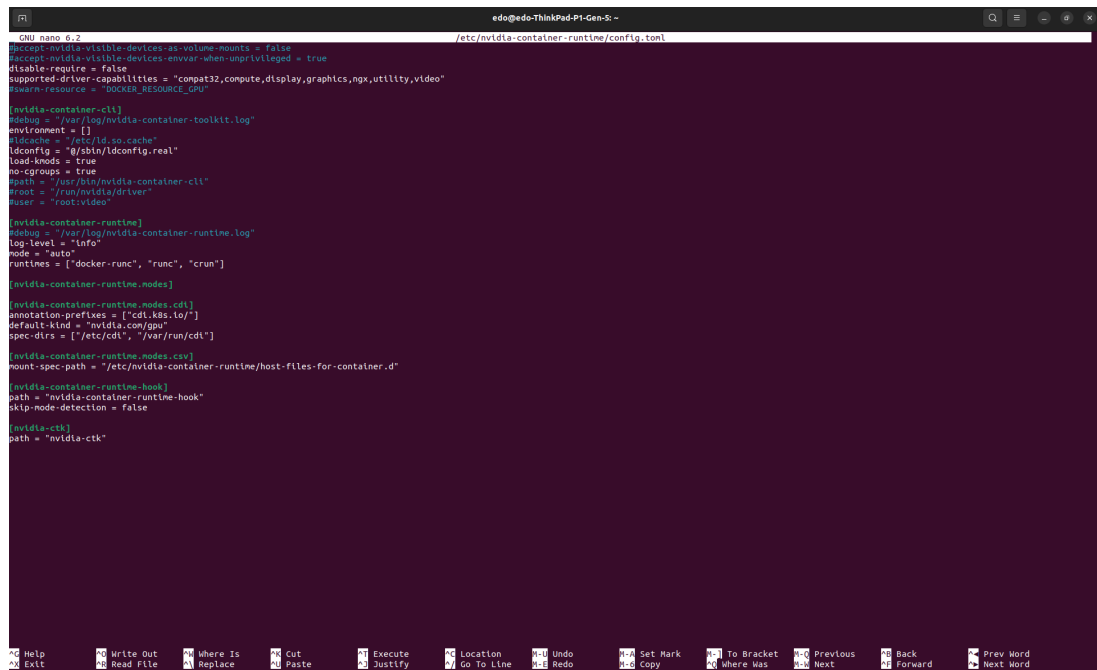
+-----+
| Processes:
| GPU   GI    CI          PID    Type    Process name
|-----+-----+
|      ID    ID
|-----+-----+
| No running processes found
+-----+
```

Complimenti, hai vinto la possibilità di andare al passaggio successivo!

2. O questo: `Failed to initialize NVML: Unknown Error` . Se sei uno dei fortunati con questo errore, allora devi fare un attimo un magheggio:

a. Apri un terminale e scrivi `sudo nano /etc/nvidia-container-runtime/config.toml`

b. Ti aprirà una schermata come questa:



```
GNU nano 2.9.2 /etc/nvidia-container-runtime/config.toml
[accept-nvidia-visible-devices-as-volume-mounts = false
[accept-nvidia-visible-devices-when-unprivileged = true
disable-require = false
supported-driver-capabilities = "compat32,compute,display,graphics,ngx,utility,video"
[swarm-resource = "DOCKER_RESOURCE_GPU"

[nvidia-container-ctk]
path = "/usr/local/nvidia-container-toolkit.log"
environment = []
[cache]
path = "/etc/ld.so.cache"
ldconfig = "/sbin/ldconfig.real"
load-kmods = true
no-cgroups = true
path = "/usr/lib/nvidia-container-ctk"
root = "/run/nvidia/driver"
user = "root:video"

[nvidia-container-runtime]
path = "/usr/local/nvidia-container-runtime.log"
log-level = "info"
mode = "auto"
runtimes = ["docker-runc", "runc", "crun"]

[nvidia-container-runtime-nodes]

[nvidia-container-runtime-nodes.cdi]
annotation-prefixes = ["cdi.k8s.io/"]
default-kind = "nvidia.com/gpu"
spec-dirs = ["/etc/cdi", "/var/run/cdi"]

[nvidia-container-runtime-nodes.csv]
mount-spec-path = "/etc/nvidia-container-runtime/host-files-for-container.d"

[nvidia-container-runtime-hook]
path = "nvidia-container-runtime-hook"
skip-node-detection = false

[nvidia-ctk]
path = "nvidia-ctk"
```

Per muoverti su nano devi usare le freccettine!

- c. Imposta `no-cgroups = false`
- d. Ora esci facendo `CTRL+S` e `CTRL+X`
- e. Ora lancia sul terminale:

- i. `sudo systemctl restart docker`

```
sudo docker run --rm --gpus all nvidia/cuda:11.0-base nvidia-smi
```

- f. Complimenti puoi continuare la guida!

3. Se hai un altro errore diverso non so che dirti, non mi è mai capitato! Prova a cercarlo su google, il mondo è bello perchè è vario!

3) Scaricare la nostra REPO ufficiale

A questo punto vai sulla nostra repo ufficiale su GitHub:

https://github.com/SmartDrive-UniPi/SmartDrive_project

Scaricala facendo: `git clone --recurse-submodules git@github.com:SmartDrive-UniPi/SmartDrive_project.git`

Ora ti basterà andare dentro la cartella `SmartDrive_project/scripts` e lanciare in sequenza:

```
bash build_docker.sh # creerà un'immagine Docker con tutto il
bash start_sim.sh # avvierà e ti farà entrare dentro al conta
```

Nota di servizio: avviando il container non si aprirà un terminale speciale o qualcosa di riconoscibile, vedrai semplicemente cambiare il nome utente dal tuo a `ubuntu`. Ti lascio di seguito una lista di comandi utili:

1. Dentro al container

- a. `CTRL+D` fatto dentro al container ti fa uscire da esso

2. Fuori dal container:

- a. Capire nome del container: `docker ps -a` ti mostrerà una lista dei container
- b. Capire immagini docker: `docker images`
- c. Riavviare il container: `docker restart <nome_container>`
- d. "Spegnere" il container: `docker stop <nome_container>`
- e. Accendere il container: `docker start <nome_container>`
- f. Entrare nel container: `docker exec -it <nome_container> /bin/bash`

Una volta dentro al container vi troverete in:

```
edo@edo-ThinkPad-P1-Gen-5:~$ docker exec -it psd_container /b
ubuntu@edo-ThinkPad-P1-Gen-5:~/psd_ws$
```

A questo punto andate nella cartella `startup/` e lanciate `bash tmuxinator.sh` .
Questo script compilerà tutti i nodi ros2 ed eseguirà un sottoinsieme di questi.

Come funziona tmuxinator.sh?

```
ubuntu@edo-ThinkPad-P1-Gen-5:~/psd_ws/startup$ cat tmuxinator.sh
#!/bin/bash
set -e

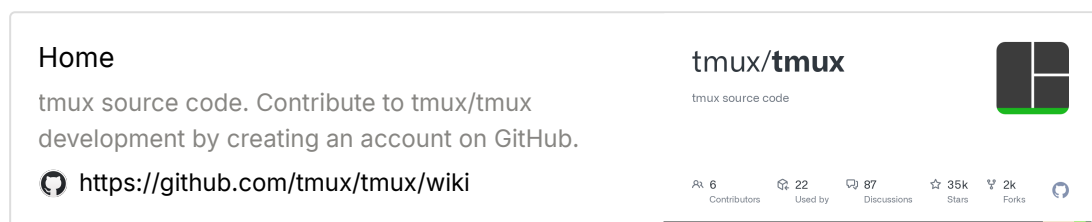
# Get the absolute path of the script and navigate to its dir
SCRIPT=$(readlink -f "$0")
cd "$(dirname "$SCRIPT")"

# Perform a colcon build in the ~/psd_ws/ directory
echo "Running colcon build in ~/psd_ws/"
cd ~/psd_ws/
colcon build --symlink-install --parallel-workers $(nproc)
echo "colcon build completed"

# start tmuxinator
cd "$(dirname "$SCRIPT")"
tmuxinator start -p session_custom_sim.yml
```

Al di là dei magheggi iniziali, quello che fa è:

1. `colcon build --symlink-install --parallel-workers $(nproc)` : compila i nodi con tutti i core disponibili creandone un link simbolico (cosa utile per poter modificare i nodi python senza dover ricompilare ogni volta)
2. `tmuxinator start -p session_custom_sim.yml` lancia una sessione di `tmux` costruita secondo le specifiche del file `session_custom_sim.yml` .
 - a. **TMUX** è un multiplexer di terminale che permette di visualizzare molteplici nello stesso.



```

ubuntu@edo-ThinkPad-P1-Gen-5:~/psd_ws/startup$ cat session_cu
name: psd_simulator
root: ./
startup_window: psd_sim_with_teleop
pre_window: |
    source ~/.bashrc;
    source /home/ubuntu/psd_ws/install/setup.bash
windows:
  - psd_sim_with_teleop:      #tmux list-windows
    panes:
      - ros2 launch psd_vehicle_bringup gz_sim.launch.py
      - ros2 run rviz2 rviz2 -d /home/ubuntu/psd_ws/startup
      - ros2 run custom_teleop custom_teleop_keyboard
      - ros2 run psd_perception fake_perception
      - ros2 run psd_slam fake_slam

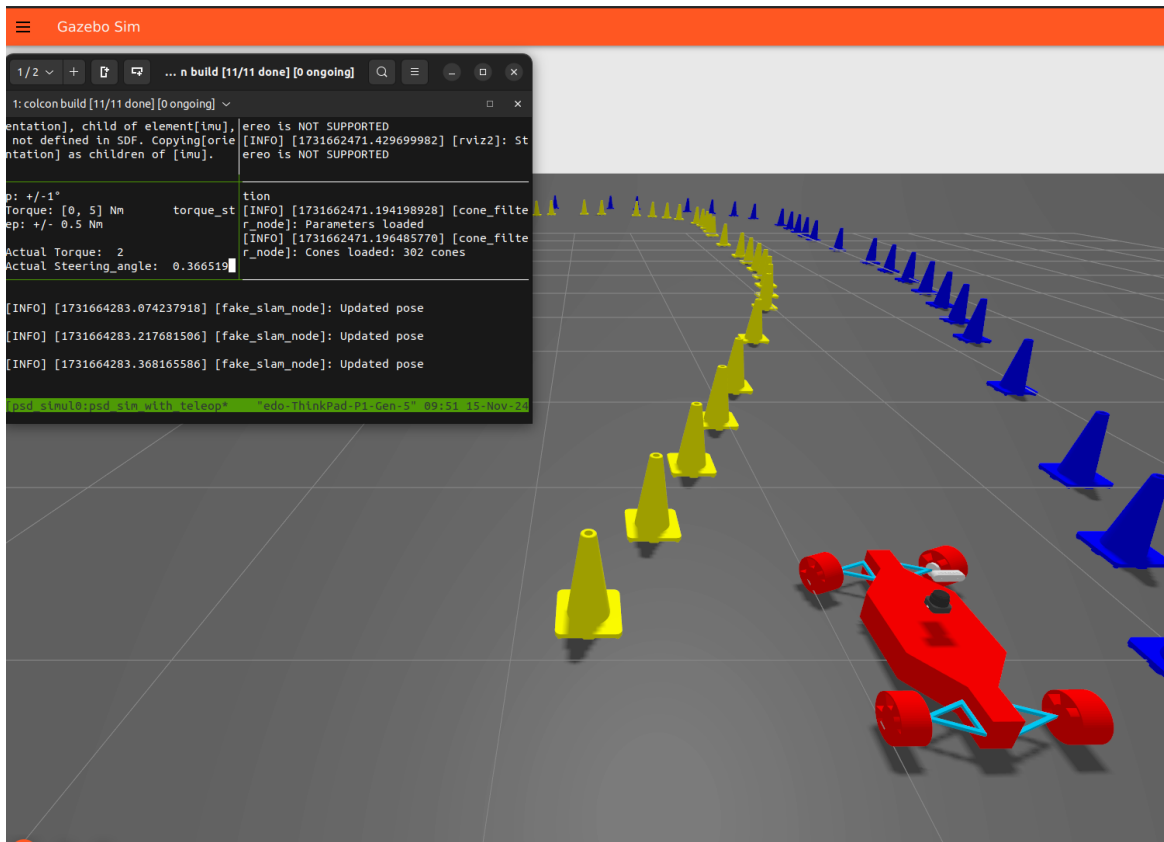
```

Quello che va a fare è:

1. Dire a tmux di creare un terminale nominato **psd_simulator** che ha base nella home
2. Prima di ogni comando successivo, fa il `source ~/.bashrc;`
`source /home/ubuntu/psd_ws/install/setup.bash`
 così da trovare ROS2 e tutti i nodi compilati in precedenza
3. Creare tanti sotto terminali spaziatati uniformemente che lancino i vari nodi di interesse, nello specifico:
 - a. `ros2 launch psd_vehicle_bringup gz_sim.launch.py` : lancia il simulatore con il veicolo
 - b. `ros2 run rviz2 rviz2 -d /home/ubuntu/psd_ws/startup/rviz/psd_vehicle.rviz` : lancia Rviz con impostazioni predefinite per visualizzare il nostro veicolo
 - c. `ros2 run custom_teleop custom_teleop_keyboard` : lancia il controllo da tastiera
 - d. `ros2 run psd_perception fake_perception` lancia il nodo che compie una 'finta' percezione (placeholder per quello definitivo, ma che restituisce gli stessi output)
 - e. `ros2 run psd_slam fake_slam` : va ad emulare anch'esso l'output della slam

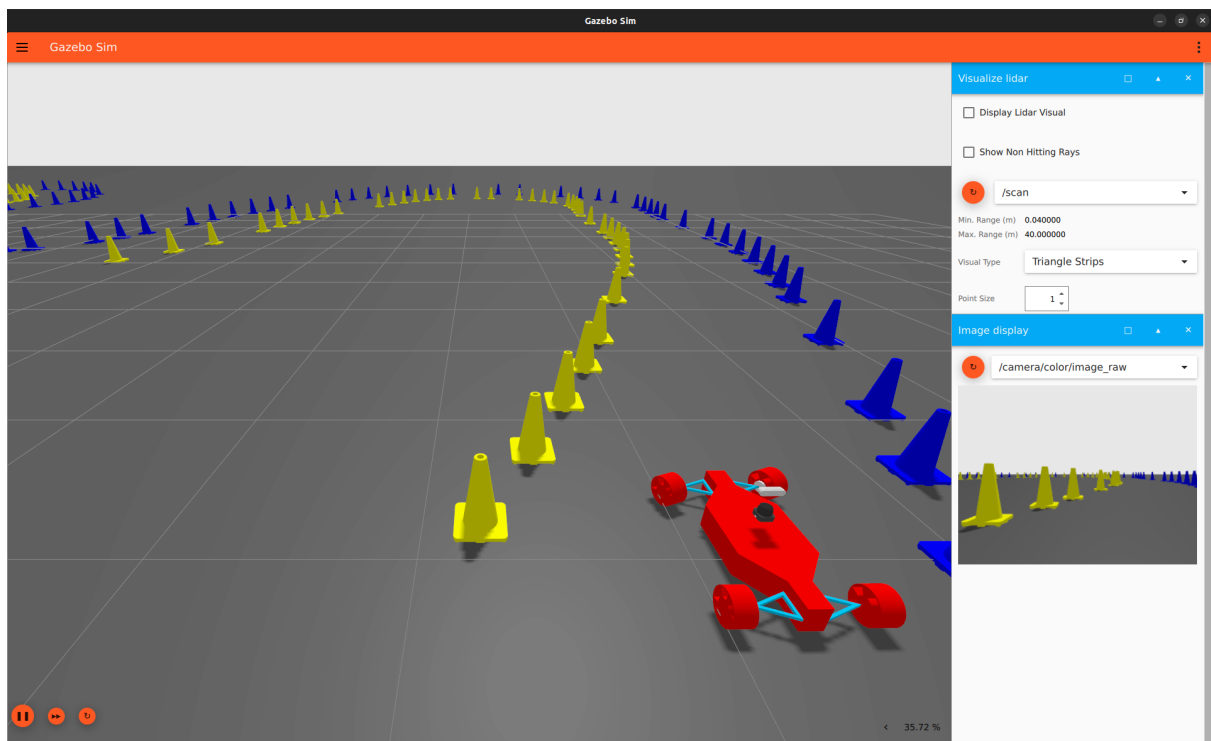
[illegible]

Per poter controllare il veicolo (ad ora) è necessario che voi siate nel terminale del `custom_teleop` e teniate il terminale in evidenza per tutta la durata del controllo. Così:



Capirete che è corretto se il rettangolino bianco è pieno e posizionato accanto al valore dell'ActualSteering!

4) Simulatore avviato!



In questa schermata vedete tre sezioni principali:

1. La finestra dove vi viene mostrata la simulazione in cui potete muovervi trascinando con il mouse e zommare con la rotella del mouse.
2. La colonna di destra con tutti gli add-on di visualizzazione attivi, in particolare nel nostro caso:
 - a. **Visualize lidar**: dove potete selezionare se mostrare i raggi del lidar o meno
 - b. **Image display** dove visualizzerete l'immagine vista dalla camera.

Al primo avvio non vedrete niente in questa colonna: vi basterà premere sul pulsante RICARICA (il cerchio arancione) accanto ai menù a tendina che indicano il topic